

Calvin Tallent and Owen Moore
May, 9, 2026

Equal contributions between Calvin and Owen

1 Introduction

While a rock, paper, scissors robot might seem benign in its use case at first, it has the potential to show us some interesting aspects of human behavior and our body's internal perception of randomness. In "A Meta-Analysis in Human Behavioral Research", Summer Armstrong notes that "An overwhelming number of experiments indicate that humans fail to produce random responses even when instructed and motivated to do so." Our group, aiming to exploit and explore this behavior to train a model that could eventually predict the behavior of the human opponent by learning their perception of 'random'. We of course, want to know if this behavior is exploitable at all by a model, and if it can effectively be done to win at rock, paper, scissors.

The personal motivations for this project lie partly in the desire to create show human random bias, but also to create an application of machine learning that was fun and more approachable than a standard data analysis project.

The challenges in this project are numerable. To start, no data on rock, paper, scissors could be found, requiring us to create a data collector. We also needed to not just analyze the data, but implement a method for our model to apply that data back into the game. We would also have to create the game.

2 Data Mining Task

The data mining task for this project involved understanding and predicting human behavior in the game of rock-paper-scissors. Since no existing datasets were available, we had to design and implement a custom data collection system to gather input from human players. This data served as the foundation for analyzing patterns in decision-making and training a predictive model. The ultimate goal was to determine whether human choices exhibit exploitable biases and to create a model capable of leveraging these biases to improve the computer's performance in the game.

We began by implementing a basic Aprori-based algorithm to find frequent item sets. From there, we were able to use the data to figure out the most frequent item. After evaluation, we determined that its performance was not great, so we implemented a new algorithm with more sophisticated methods,

The amount of games the user will be prompted to complete can be changed within the program, but in general, they will be required to play enough that sufficient data can be gathered. Once this data is collected, it will be fed into the Apriori algorithm to determine sets of frequent items. The data will also be passed into a simple algorithm which merely determines the most frequent item and its success rate.

Using this data, we adjust the odds of certain options being picked, by adjusting the values in the array of odds that we stored earlier, to reflect the outcome of these algorithms. In general, we want to adjust down the likelihood of the computer picking a bad option, and adjust up the odds of the computer picking a good option.

This does present a slight challenge, in that the odds are out of 1. Therefore, adjusting one option means the others must be adjusted. Also since Apriori sets of frequent items, it will potentially leave us with many things to adjust.

Aside from these minor challenges, the main challenges of this project will include implementing these algorithms, and effectively implementing a persistent form of data collection.

The main data mining question at hand is can we effectively use previous gameplay to predict the choices of a human player.

3 New Algorithm

The general architecture of this algorithm is a probabilistic choice system. For each move, it employs a number of "experts" — subroutines that attempt to predict what the opponent will do in a variety of ways. The main algorithm takes all that information, weighted over time, and uses it to pick the best move.

The various "experts" are below:

- "freq" — looks at the overall tendency that the opponent has played each of rock, paper, scissors (i.e. just a global count, which decays over time)
- "markov1" — uses Markov chains of length 1 to store a list of connected moves. For example, if the opponent has played rock then scissors before, him playing rock now will cause markov1 to predict scissors.
- "markov2" — Uses Markov chains of length 2, very similar to markov1.

- "outcome" — uses the outcome of the game to predict what will be played. For instance, if the opponent frequently plays scissors after winning, it would predict scissors after winning games.
- "copy" — predicts the opponent will copy your last move
- "anticopy" — predicts the opponent will play the counter to your last move

For every game, the main algorithm calls each of the experts, gets their prediction, then weights it by time (so that newer games are weighted more heavily than older ones). Once it has the weighted prediction (i.e. a probability distribution of the three options), the algorithm randomly, according to the distribution, plays one of the three options.

4 Technical Approach

In a technical sense, our program is composed of two different approaches at solving this same problem. That being a simple "Most Popular Item" method, and an "Aggregate Expert" approach. More detail of which will be discussed further in this article.

When the program is started, we first have to load the previous sessions data to keep a persistent memory. This accomplished by using Python pickles. Pickles are a simple way to store binary data between sessions. We use it to store a list containing each game, which will be discussed more later.

Next, the user is prompted to play a game or rock, paper, scissors. The computer is set initially to pick completely randomly. This is done by storing the likelihood of each option being picked in an array with 3 values which encode the odds of a specific option being picked.

Right away, we want to create a list the win/loss ratio, this is also where we create our list of game data that we load from a pickle as mentioned previously. The win/loss ratio is our main data metric to track how good the program is doing against the human player. The idea is that the higher the win/loss ratio, the better it is at predicting the human's behavior.

We also want to define the array, from which the computer gets its random odds from, as mentioned earlier.

Then, we start our main game loop. The program runs on a system of rounds and games, where there is a certain number of "games" played in a

certain number of “rounds”. The default for these values in our program is 5 games per round, and 200 rounds.

As the games are played their results are stored in this data structure:

In English, all this is a list of a datatype we call a “GameType”, which is a 3×2 numpy array, which stores binary values. Each player is assigned a row, which contains 3 of these binary values. These values encode the choice of that player in that round. Essentially, one-hot encoding the choice. The GameType wraps these two choices in the numpy array, and these numpy arrays are stored in what is called a gameSeries.

The gameSeries represents each round of play, and is the level of data where win/loss is calculated, and also the level on which the “Most Popular Item” approach is implemented.

The way this algorithm works, is it loops through all the games in the game series, and uses a hash function to match those games with a pre-determined game type (since there’s only so many types of possible games in rock, paper, scissors). From this, we can determine some information about the game, including, most important to us in this method, whether the most frequent game in the last round was a “winning”, or a “losing” game.

If the game is determined to be a “winning” game, we increase the odds of the computer able to roll that move, by a factor of the “learning rate”, which doesn’t function like a typical learning rate, and instead, is simply the number of percentage points we add or subtract from the likelihood that number gets picked. The other two numbers in the array are decreased or increased, in equal ammounts, opposite of the direction the most frequent item went.

After all the games are played, and the win/loss ratio to view the data. We also need to remember here to store our data back into the Python pickle to maintain persistence.

5 Evaluation Methodology

The primary method for evaluating our approach is simply the win/loss rate. When playing against a truly random opponent, one would expect the win/loss rate to be close to 50%; when playing against a human or other non-random opponent, the win/loss rate should hopefully be much higher (depending on what variables the opponent is taking into account).

6 Results and Discussion

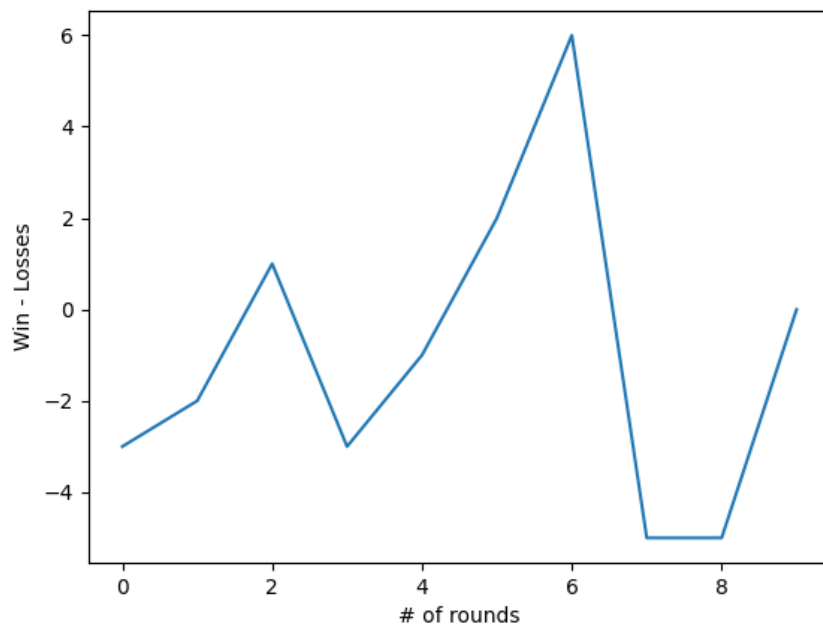
As mentioned before, we had originally planned to implement the Apriori algorithm to be able to get multiple most frequent itemsets. However, this proved to be not only very difficult to implement, due to the shape of our data, but also unnecessary.

As the project went on, it became increasingly difficult to justify the time taken to get the algorithm to work, due to its limited perceived success compared to a simpler system that only retrieved the most frequent item in a set. Apriori gets the most frequent combinations of itemsets, but this information seemed irrelevant, so we decided to abandon this method in favor of our simpler approach.

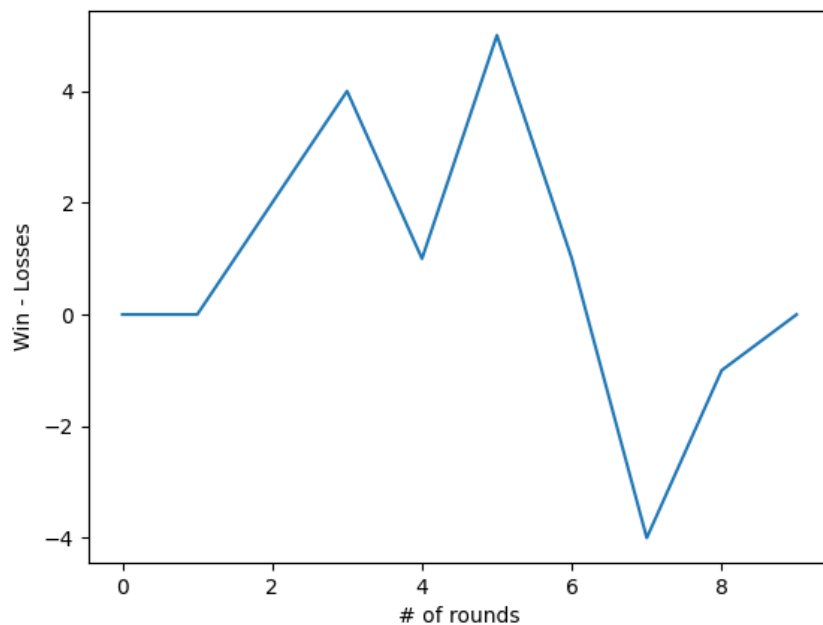
What we discovered, was that the simple model, over short courses, and especially against another computer that only played random, did terribly. However, the “Aggregate Expert” model fared much better.

The “Most Popular Item”, going up against another computer who chose random every time, saw limited success.

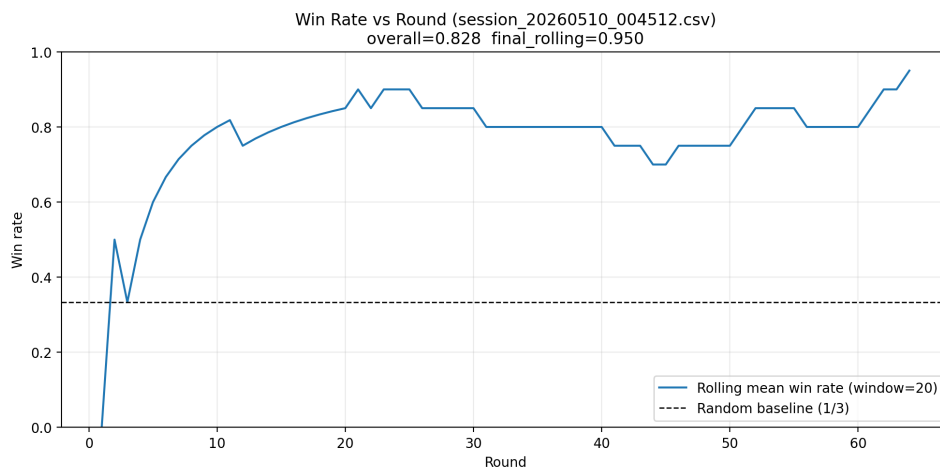
The algorithm proved to not work very well, most likely due to the simplicity of the model mixed with the relatively low number of rounds. The experiments for this model were done with the games per round set to 20, the number of rounds set to 10, and the learning rate set to 0.2, as in 20



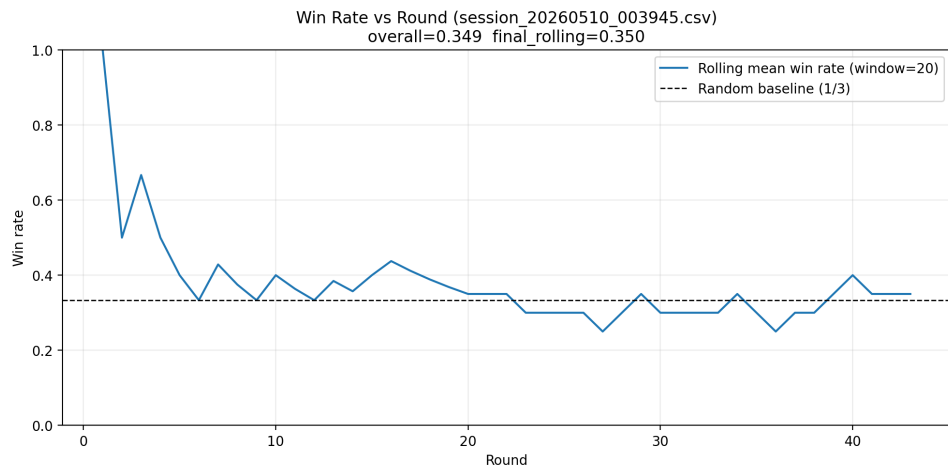
Out of curiosity, to see how it faired with a more controlled environment, I decided to do a few rounds of only playing paper against the machine, and was given this as a result.



Now, for the “Aggrigate Expert” algorithm, which in a similar paper-only experiment, faired much better.



Due to the success of this algorithm in getting much closer to the ideal outcome, it was tested with real human semi-random input, achieving this.



References:

Armstrong, Summer Ann, "A meta-analysis of randomness in human behavioral research" (2004). LSU Master's Theses. 284.

https://repository.lsu.edu/gradschool_theses/284